

Not All Instructions Are Forgotten Equal

Tomás P. Korenblit 

Universidad Nacional de San Martín, Buenos Aires, Argentina
tomaskorenblit@gmail.com

Abstract. Large language models lose adherence to user-stated preferences over long conversations. This holds even when the system prompt keeps its other instructions fixed, so the loss has no visible trigger. Our central finding is that they do not forget these preferences equally: retention is markedly uneven from one instruction to the next, and treating it as uniform hides that variation. Prior work reports mean compliance and mitigates uniformly, restating every instruction each turn, which assumes all instructions are retained alike; we measure retention per instruction instead. We plant preferences in realistic multi-turn coding sessions on three open-source Python codebases, across several models and twelve instruction types, and grade compliance with deterministic checkers rather than an LLM judge, so the scores are reproducible. A Bayesian hierarchical model then estimates how strongly each instruction is retained. Most preferences are already followed by default, a few are honored only when explicitly restated, and one shows a small negative estimate that is likely a measurement artifact. Only three of twelve instruction types gain enough from restating to justify it, which argues for targeted rather than blanket reinforcement. The asymmetry matters most for agents deployed in long-running loops, where no human reviews each step: guardrails are themselves instructions, so a safety-relevant rule can decay silently while the agent keeps acting, and without per-instruction monitoring nothing flags it.

Keywords: instruction retention , context rot , Bayesian inference , AI safety , LLM evaluation

No Todas las Instrucciones se Olvidan Igual

Abstract. Los grandes modelos de lenguaje pierden adherencia a las preferencias del usuario en conversaciones extensas. Esto ocurre incluso cuando el system prompt mantiene fijas sus demás instrucciones, de modo que la pérdida no tiene señal visible. Nuestro hallazgo central es que no se olvidan todas las preferencias por igual: la retención es marcadamente desapareja de una instrucción a otra, y tratarla como uniforme oculta esa disparidad. El trabajo previo reporta cumplimiento promedio y mitiga

uniformemente, repitiendo cada instrucción en cada turno, asumiendo que todas se retienen igual; nosotros medimos la retención por instrucción. Plantamos preferencias en sesiones de programación multi-turno sobre tres repositorios de Python de código abierto, en varios modelos y doce tipos de instrucción, y evaluamos el cumplimiento con verificadores determinísticos en lugar de un juez LLM, de modo que los puntajes son reproducibles. Un modelo jerárquico bayesiano estima cuánto se retiene cada instrucción. La mayoría de las preferencias ya se cumplen por defecto, unas pocas se respetan solo al enunciarse, y una muestra un pequeño efecto negativo, probablemente un artefacto de medición. Solo tres de doce tipos de instrucción ganan lo suficiente al reenunciarse para justificarlo, lo que favorece un refuerzo dirigido antes que uniforme. La asimetría importa sobre todo para agentes desplegados en loops, donde ningún humano revisa cada paso: las barreras de seguridad son también instrucciones, de modo que una regla relevante para la seguridad puede degradarse en silencio mientras el agente sigue actuando, y sin monitoreo por instrucción esa degradación pasa inadvertida.

Keywords: retención de instrucciones , degradación de contexto , inferencia bayesiana , seguridad de la IA , evaluación de LLMs

1 Introduction

Language models are increasingly deployed as autonomous agents that run in loops, call tools, and act over hundreds of turns with no human reviewing each step. An agent is governed by its instructions: the operational limits and safety guardrails it is given, usually once and at the start, that must hold for the whole run. Yet adherence to instructions erodes as the context grows, a phenomenon we refer to as *context rot*. Laban et al. (2025) measured average compliance drops of 39% across 15 models in multi-turn settings. The cause is architectural: attention is not uniformly distributed across the context window, and information in middle positions receives less weight than at the boundaries (Chowdhury, 2026; Liu et al., 2024). An instruction stated once and then buried under accumulating tool output competes for attention against everything that follows it, and in a deployed loop there is no user to notice the lapse and restate it.

This is a safety concern before it is a cost concern. When the model drops a preference, a chat user notices and restates it; a deployed agent has no such supervisor, so a guardrail that decays does so silently while the agent keeps acting through tools whose effects are real and often irreversible (Mu et al., 2025). Repeating every instruction each turn, the standard mitigation, multiplies prompt cost and still cannot tell an operator which constraint remains in force. Because safety is conjunctive, an aggregate compliance rate says nothing about whether the one rule that matters is among those that lapsed.

No prior work has measured this heterogeneity at the level of individual instructions. Existing studies report mean compliance across instruction sets (He et al., 2024; Laban et al., 2025) or propose uniform mitigation strategies such as periodic reminders (Dongre et al., 2025). But if instructions are retained unequally, the mean hides the per-instruction differences a reinforcement policy would act on. An instruction that is already retained does not need reinforcement, and one that is harmed by repetition should not receive it; without per-instruction estimates, a policy cannot tell these cases apart.

Bayesian Knowledge Tracing (Corbett & Anderson, 1994) offers a framework for exactly this kind of estimation. Originally developed in education, BKT estimates the probability that a student has mastered a concept based on a sequence of observed responses. We adapt this: the LLM is the student, and its instructions are the concepts. A BKT-based reinforcement system would only be useful if instruction retention is in fact heterogeneous. We test this using a Bayesian ordered logistic model with hierarchical effects per decision type, fitted to compliance scores collected from realistic multi-turn coding conversations across three open-source codebases.

We report three findings. First, retention heterogeneity across instruction types is large, with treatment effects on the log-odds scale ranging from -2.4 to $+5.4$ (group-level standard deviation $\sigma_\beta = 2.1$). Second, one instruction type (`dependencies_no_new`) shows a marginal negative effect (its 94% HDI barely excludes zero), which we read as a likely measurement artifact rather than a robust harm. Third, within the resolution of this design, the model and codebase size explain little variance in retention; because baseline data comes from a single model, we read this as model-invariance within our design rather than an established absence of model effects. A Bayesian model can then identify which instructions to reinforce and which to leave alone.

2 Related work

Instruction compliance degradation. Several recent studies have established that LLMs lose adherence to instructions as conversations grow. Laban et al. (2025) documented an average 39% compliance drop across 15 models in multi-turn settings over 200K conversations. He et al. (2024) showed that accuracy on multi-instruction tasks drops from 87.7% to 70.7% within three turns, even when instructions are given simultaneously. Du et al. (2025) demonstrated that increasing the context window alone does not resolve this degradation. All three studies report aggregate metrics across instruction types. None examines whether different instructions are retained to different degrees.

Positional attention and prompt saturation. The mechanism behind compliance degradation has roots in the attention architecture itself. Liu et al. (2024) identified a U-shaped attention curve: information at the beginning and end of the context receives more weight, while middle positions are underweighted. This positional bias explains why system prompt instructions, which occupy position

zero, remain more stable than user-stated preferences buried mid-conversation. Mu et al. (2025) showed a related saturation effect: as the number of guardrails in a system prompt increases, compliance with any individual rule drops sharply. Adherence therefore degrades unevenly: it depends on where an instruction sits and how many others surround it.

Uniform mitigation strategies. Existing approaches to maintaining compliance treat all instructions equally. Dongre et al. (2025) modeled instruction drift as a bounded stochastic process and proposed periodic reminders to reduce divergence. Leviathan et al. (2025) studied the effect of prompt repetition on instruction stability. Both approaches reinforce all instructions at each intervention, regardless of whether an individual instruction has been retained or forgotten. This uniform strategy becomes expensive at scale and, as our results show, can be counterproductive for instructions where repetition degrades compliance.

Bayesian Knowledge Tracing. BKT has tracked per-concept mastery probabilities in education for over 30 years (Corbett & Anderson, 1994). The per-instruction heterogeneity that makes BKT useful there also appears in LLM instruction compliance, with each instruction in the role of a concept.

3 Method

3.1 Decision planting protocol

We simulate an AI coding assistant working on a real open-source codebase over 25 turns. At each turn, the assistant receives a user message and may call tools (read files, write code, search the codebase). Real source files are injected into the conversation at turns 0–19, building up context pressure as the conversation progresses.

In the **treatment** condition, 12 coding preferences are embedded as casual inline requests within the user’s messages at specific turns. Six are planted early (turns 1–7) and six late (turns 14–19). Each preference targets a different coding practice: code style (NumPy broadcasting, list comprehensions), architecture (subclassing vs. standalone functions), testing (parametrize, approximate assertions), naming (underscore prefix, no abbreviations), dependencies (no new imports, module-level constants), and documentation (NumPy-style docstrings, regex comments). The instructions are phrased as natural user preferences, not system prompt rules (e.g., “Important: when writing array operations for this project, always use NumPy broadcasting instead of for loops”).

In the **baseline** condition, the conversation follows the same structure with the same file injections and the same task, but no preferences are planted. This allows us to measure what the model does by default for each decision type.

At turns 20–25, the user requests code generation tasks designed to elicit each decision. Each test turn evaluates exactly two decisions (one early-planted, one late-planted), and the model’s code output is scored by deterministic checkers.

3.2 Compliance measurement

Each of the 12 decision types is evaluated by a deterministic checker that parses the model’s code output using regular expressions and pattern matching. Checkers are fully automated with no LLM-in-the-loop evaluation. This makes results reproducible across runs. Each checker produces an ordinal score on a 0–3 scale:

- **0**: Completely ignored or violated
- **1**: Minimal or inconsistent compliance
- **2**: Mostly followed with minor deviations
- **3**: Fully followed

For example, the `parametrize` checker searches for `pytest.mark.parametrize` decorators and penalizes separate test functions. The `broadcasting` checker detects `for` loops and rewards NumPy operations. The `no_new_imports` checker counts `import` statements in generated code.

3.3 Bayesian model

We model the ordinal compliance scores using an ordered logistic regression (cumulative link model) with hierarchical effects per decision type. The ordered logistic is appropriate because the 0–3 scores have unequal intervals: the gap between 0 (ignored) and 1 (minimal compliance) is qualitatively different from the gap between 2 (mostly followed) and 3 (fully followed). A linear model would incorrectly assume equal spacing.

For each observation i , the latent compliance η_i is:

$$\eta_i = \alpha_{d[i]} + \beta_{d[i]} \cdot \text{treatment}_i \quad (1)$$

where $d[i]$ indexes the decision type, α_d is the baseline intercept (compliance without being told), and β_d is the treatment effect (how much telling improves compliance). The observed score is determined by where η falls relative to three ordered cutpoints $c_1 < c_2 < c_3$.

Both α and β are given hierarchical priors with non-centered parameterization:

$$\alpha_d = \mu_\alpha + \sigma_\alpha \cdot \tilde{\alpha}_d, \quad \tilde{\alpha}_d \sim \text{Normal}(0, 1) \quad (2)$$

$$\beta_d = \mu_\beta + \sigma_\beta \cdot \tilde{\beta}_d, \quad \tilde{\beta}_d \sim \text{Normal}(0, 1) \quad (3)$$

The hyperpriors are weakly informative on the log-odds scale: $\mu_\alpha \sim \text{Normal}(0, 1.5)$, $\mu_\beta \sim \text{Normal}(0, 1)$ (centered at zero, no prior assumption about direction), $\sigma_\alpha, \sigma_\beta \sim \text{Exp}(1)$. Cutpoints use an offset parameterization: $c_1 \sim \text{Normal}(0, 1.5)$ with positive increments $\Delta c \sim \text{Exp}(1)$ to guarantee ordering.

The key scientific parameter is σ_β : the group-level standard deviation of treatment effects. A large σ_β indicates that different instructions respond differently to being stated, which is the premise required for selective reinforcement.

We initially fit a full model including per-model and per-codebase intercepts. Both group-level standard deviations were small, with posteriors concentrated near zero and 94% intervals spanning zero ($\sigma_{\text{model}}, \sigma_{\text{codebase}} \approx 0.3$), so these terms add no explanatory power. We therefore report the simpler per-decision model, whose diagnostics are clean (zero divergences, all $\hat{R} \leq 1.01$, minimum ESS = 1311). With baseline data from a single model (Section 6.3), per-model effects are in any case weakly identified.

The model was implemented in PyMC (Abril-Pla et al., 2023). Posterior samples were drawn using the No-U-Turn Sampler (NUTS; Hoffman & Gelman, 2014) via the nutpie sampler, a Rust-based NUTS implementation (Seyboldt, 2024), with 4 chains of 1000 draws each following 1000 tuning steps. Prior predictive checks confirmed that the priors produce plausible score distributions before fitting. Posterior predictive checks verified that the fitted model reproduces the observed data.

4 Experimental setup

4.1 Codebases

We selected three open-source Python libraries from the Bayesian statistics ecosystem, chosen to span a range of codebase sizes and thus context pressure levels:

- **Bambi** (~10K lines): a high-level interface for Bayesian regression. By turn 20, injected files produce approximately 98K tokens of context.
- **ArviZ** (~25K lines): a library for exploratory analysis of Bayesian models. Context reaches approximately 160K tokens by turn 20.
- **PyMC** (~57K lines): a probabilistic programming framework. Context reaches approximately 203K tokens by turn 20.

All repositories were cloned at fixed commits: Bambi (5110615, 2026-03-17), ArviZ (arviz-base at 374cd28 and arviz-stats at 8d6316a, 2026-03-26/29), and PyMC (bca2b1e, 2026-03-27, branch v6). Source files were injected into the conversation at turns 0–19 via the tool execution system, which serves real file contents from the cloned repositories.

4.2 Models

We evaluated five instruction-following models across two experiment phases. Table 1 summarizes the models and conditions.

Qwen served as the primary model with both baseline and treatment conditions across all three codebases (18 conversations). The remaining four models were tested in the treatment condition only, to verify that the retention patterns generalize across architectures. Gemma was tested on Bambi only due to budget constraints.

Table 1. Models and experimental conditions. Only Qwen was tested in both baseline and treatment conditions.

Model	Parameters	Baseline	Treatment
Qwen 3.5	27B	✓	✓
Gemini 3.1 Flash Lite	–		✓
GPT-5.4-nano	–		✓
Nemotron 3 Super	120B		✓
Gemma 4	26B		✓

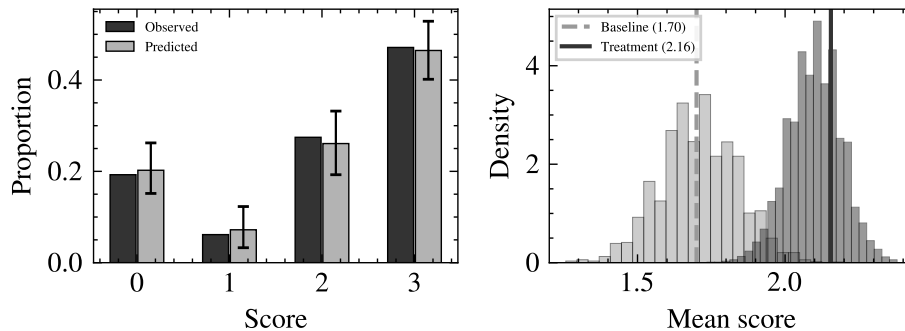


Fig. 1. Posterior predictive check. Left: the model reproduces the observed score distribution across all four ordinal categories. Right: the model captures the difference in mean compliance between baseline (1.70) and treatment (2.16) conditions.

4.3 Dataset

We ran 28 conversations in total: 9 baseline and 19 treatment. Models were accessed through OpenRouter (Qwen, Gemini, Gemma, Nemotron) and the OpenAI API (GPT-5.4-nano). Not every conversation produced a scorable response at every test turn; the final dataset consists of 244 compliance observations: each observation is one decision type scored at one test turn in one conversation. Observations span 5 models, 12 decision types, 3 codebases, and 2 conditions. The baseline condition contributes 70 observations (all from Qwen), and the treatment condition contributes 174 observations across all five models. All conversation logs, checker code, and analysis notebooks are available in the supplementary material.

5 Results

The model converged cleanly: zero divergences, $\hat{R} \leq 1.01$ for all parameters, and a minimum effective sample size of 1311. Posterior predictive checks confirm that the model reproduces the observed score distribution (Figure 1).

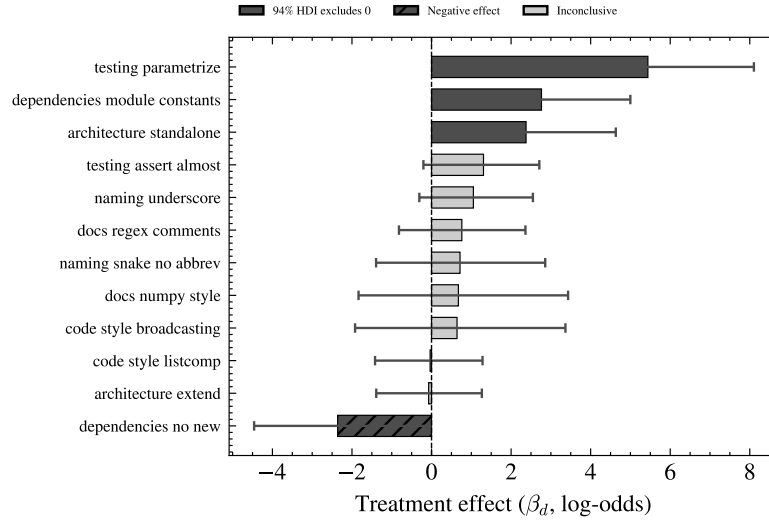


Fig. 2. Per-decision treatment effects (β_d) with 94% HDI. Dark bars: 94% HDI excludes zero. Hatched: negative effect. Light bars: inconclusive.

5.1 Treatment effect heterogeneity

The group-level standard deviation of treatment effects is $\sigma_\beta = 2.11$ (94% HDI: [1.06, 3.28]), indicating that different instruction types respond differently to being stated. Individual treatment effects on the log-odds scale range from -2.36 to $+5.44$ (Figure 2).

The posterior separates the 12 decision types into three groups:

Instructions that benefit from reinforcement. Three decision types have treatment effects with 94% HDI entirely above zero. `testing_parametrize` has the largest effect ($\beta = 5.44$, HDI: [2.94, 8.10]): models almost never use `pytest.mark.parametrize` unprompted (baseline mean: 0.00) but retain the preference when told (treatment mean: 2.79). `dependencies_module_constants` ($\beta = 2.76$, HDI: [0.60, 5.00]) and `architecture_standalone` ($\beta = 2.38$, HDI: [0.29, 4.63]) follow the same pattern: near-zero baseline, meaningful retention when stated.

A marginal negative effect. The only decision type with a negative estimate is `dependencies_no_new` ($\beta = -2.36$, 94% HDI [-4.46, -0.03]), but the effect is marginal: the interval only just excludes zero. The baseline mean score is 2.25 (models naturally avoid adding imports), while stating “do not add new imports” lowers the treatment mean to 0.77. One hypothesis is that the explicit negation confuses the model’s instruction following. But the interval barely excludes zero, and the checker for this type (`no_new_imports`) counts *any* import statement as new, including re-shown imports already present in the file (Section 6.3). We therefore treat this as a candidate measurement artifact rather than a robust

harm. Isolating the negation from the content (comparing “avoid new imports” with “use only existing imports”) would settle it.

Instructions that need no intervention. The remaining eight decision types show treatment effects with HDIs crossing zero. Several of these have high baseline compliance (**broadcasting**: 3.00, **numpy_style**: 3.00, **snake_no_abbrev**: 2.67), meaning models already follow these conventions without prompting; reinforcing them does not change the score.

5.2 Model and codebase effects

As described in Section 3.3, a model with per-model and per-codebase intercepts placed both group-level standard deviations near zero, so we report the per-decision model. Retention is consistent across five models (26B to 120B for the three with disclosed sizes) and three levels of context pressure (98K to 203K tokens). Within the resolution of this design, what determines whether an instruction is retained is the instruction itself rather than the model processing it or the amount of surrounding context; we caution that the baseline condition comes from a single model (Section 6.3), so per-model retention cannot be fully separated.

5.3 Reinforcement priorities

We use the posterior to derive a reinforcement ranking. For each decision type d , we compute $P(\text{score} \geq 2 \mid \text{told})$ and $P(\text{score} \geq 2 \mid \text{not told})$ across all posterior samples, and define the reinforcement gain as their difference (Figure 3). This is not a closed-loop policy evaluation; it is a posterior-derived priority ranking that identifies which instructions would benefit from reinforcement.

Table 2 summarizes the ranking. Of 12 instruction types, only 3 have a reinforcement gain with 94% HDI entirely above zero. A uniform policy that reinforces all 12 would spend tokens on 8 instructions that need no help and 1 with a marginal negative effect.

6 Discussion

6.1 What determines retention

The treatment effects follow a pattern: instructions that ask for something the model would not do by default are retained when stated, while instructions that match existing behavior show no treatment effect. The three decisions with clear positive effects (**parametrize**, **module_constants**, **standalone**) all have baseline compliance near zero. The decisions with no treatment effect (**broadcasting**, **numpy_style**, **snake_no_abbrev**) have baseline compliance near 3.0. The models’ defaults already match these conventions, so restating them has no effect.

The negative effect on **dependencies_no_new** ($P(\beta < 0) = 0.98$) is the only negative effect in the study. The instruction “do not add new imports” is the only

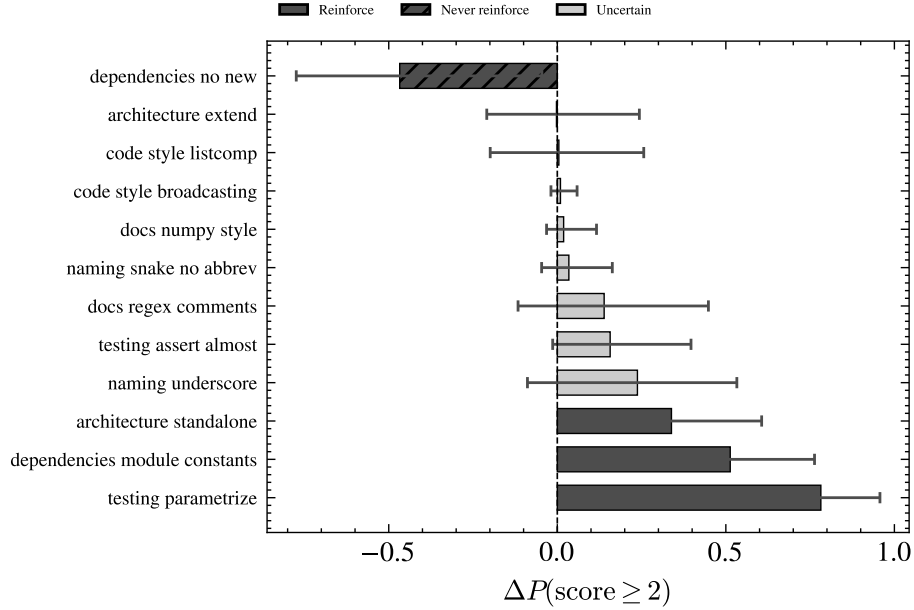


Fig. 3. Expected change in $P(\text{score} \geq 2)$ from reinforcing each decision, with 94% HDI. Dark bars: HDI excludes zero. Hatched: reinforcement is harmful.

one phrased as a negation. All other instructions tell the model what to do, not what to avoid. This raises the question of whether the negative treatment effect reflects something about the content of the instruction (import management) or its framing (negation). Which case holds determines the fix: if negation is the problem, reformulating as “use only existing imports” might produce a positive effect. If import management is inherently hard to retain, no framing will help. Distinguishing these hypotheses requires a controlled comparison that holds semantic content constant while varying the negation frame. A reviewer suggested exactly this test; we adopt it in the future work below.

6.2 Robustness checks

To address the concern that baseline data comes from Qwen only, we fit the same model restricted to Qwen observations (132 of 244, both conditions). The per-decision treatment effects correlate at $r = 0.80$ with the full-model estimates, and the group-level heterogeneity parameter remains stable ($\sigma_\beta = 2.35$ vs. 2.11 in the full model). Three of the strongest effects replicate: **parametrize** ($\beta = 6.13$), **module_constants** ($\beta = 2.25$), and **dependencies_no_new** ($\beta = -1.07$, still negative though with wider intervals).

One decision, **architecture_standalone**, does not replicate in the Qwen-only analysis ($\beta = 2.38$ in the full model vs. $\beta = -0.71$ with Qwen alone). Its

Table 2. Reinforcement priorities derived from posterior estimates. Gain is $\Delta P(\text{score} \geq 2)$ from reinforcement. The five skipped decisions are `broadcasting`, `listcomp`, `architecture_extend`, `snake_no_abbrev`, and `numpy_style`.

Decision type	Gain	94% HDI	Action
<code>testing_parametrize</code>	+0.78	[+0.46, +0.96]	Reinforce
<code>module_constants</code>	+0.51	[+0.18, +0.76]	Reinforce
<code>arch_standalone</code>	+0.34	[+0.06, +0.61]	Reinforce
<code>naming_underscore</code>	+0.24	[−0.09, +0.53]	Uncertain
<code>testing_assert_almost</code>	+0.16	[−0.01, +0.40]	Uncertain
<code>docs_regex_comments</code>	+0.14	[−0.12, +0.45]	Uncertain
5 other types	< +0.04	crosses zero	Skip
<code>deps_no_new</code>	−0.47	[−0.77, −0.05]	Avoid (marginal)

treatment effect in the full model appears driven by the other four models. We reduce the count of clearly beneficial decisions from three to two when considering only Qwen data.

Leave-one-out cross-validation shows no problematic observations (all Pareto $\hat{k} < 0.7$, $\text{ELPD} = -228.8 \pm 14.5$). Per-decision posterior predictive checks confirm that the model reproduces the observed mean score for all 12 decision types within the 90% predictive interval (Figure 4).

6.3 Limitations

Baseline data comes from one model only (Qwen). The sensitivity analysis ($r = 0.80$) mitigates but does not eliminate this concern. Future replications should include baseline conditions for every model tested.

The dataset contains 244 observations, with some decision-by-condition cells as small as 4 observations. The hierarchical model handles sparsity through partial pooling, and the wide credible intervals for low-data decisions reflect this uncertainty. However, the reinforcement ranking for the “uncertain” category (Table 2) should be treated as preliminary.

Compliance was measured at turns 20–25 only. This gives a retention snapshot, not a decay curve. We cannot estimate per-instruction decay rates or determine at which turn compliance begins to drop. Fitting actual forgetting curves would require compliance measurements at every turn, which would then require designing prompts that give the model an opportunity to demonstrate each decision at each turn without making the conversation artificial.

The deterministic checkers have construct-validity limitations. When a checker cannot find a relevant code pattern, it defaults to a score of 2 (“no clear signal”). Four decision types have default-2 rates above 50%: `architecture_extend` (79%), `docs_regex_comments` (75%), `code_style_listcomp` (72%), and `testing_assert_almost` (50%). The three decisions with the strongest treatment effects (`parametrize`, `standalone`, `no_new`) have 0% default-2 rates, so the headline findings are not driven by this artifact. The `no_new_imports` checker penalizes

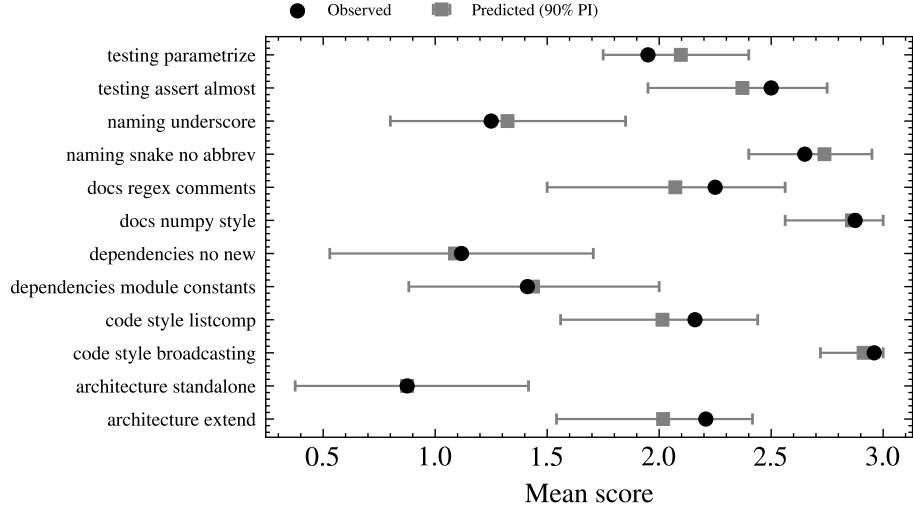


Fig. 4. Per-decision posterior predictive check. Observed means (dots) fall within the 90% posterior predictive interval for all 12 decision types.

any `import` statement, including standard library modules that may be required by the task.

This paper does not implement Bayesian Knowledge Tracing. It measures the per-instruction heterogeneity that would make a BKT-based reinforcement system useful. Building and evaluating such a system, including closed-loop token-budget allocation and real-time compliance tracking, is future work.

6.4 Future work

Several directions follow from these results and from reviewer feedback. The most direct is to collect matched baseline and treatment conditions for every model rather than for Qwen alone, which would let us estimate per-model effects directly instead of inferring model-invariance from a single-model sensitivity analysis. The deterministic checkers, though reproducible, have construct-validity limits, so we plan to pair them with a blinded panel of LLM judges from model families distinct from those under evaluation and to report checker-judge and inter-judge agreement. To separate phrasing from content, we would re-run the instructions whose treatment effect excludes zero under negated paraphrases, and the negated instruction under positive paraphrases, measuring within-instruction phrasing variance against between-instruction variance. Scoring compliance at every turn, rather than only at turns 20–25, would let us fit per-instruction forgetting curves and estimate per-instruction decay rates. Finally, implementing the posterior-derived ranking as a runtime policy would test whether selective reinforcement beats uniform reinforcement under a fixed token budget, and replacing simulated

planting with traces from real coding sessions would test whether the structure holds outside the harness.

7 Conclusion

We measured per-instruction retention in AI coding assistants and found treatment effects spanning nearly eight log-odds units, from instructions the model adopts only when told, to ones it already follows by default. The pattern holds across five models and three levels of context pressure, and the heterogeneity replicates on a single model with matched baseline and treatment conditions ($r = 0.80$); with baseline data from one model, though, we cannot fully separate per-model from per-instruction effects.

The practical payoff is concrete: of 12 instruction types only 3 show a reinforcement gain whose interval excludes zero, so a uniform policy spends most of its tokens where they buy nothing. The retention probabilities estimated here are the input a Bayesian Knowledge Tracing system would need; we leave building it to future work.

Use of large language models During the preparation of this work the author used a large language model (Claude) in order to improve the language and readability of the manuscript.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

- Abril-Pla, O., Andreani, V., Carroll, C., Dong, L., Fonnesbeck, C. J., Kochurov, M., Kumar, R., Lao, J., Luhmann, C. C., Martin, O. A., Osthege, M., Vieira, R., Wiecki, T., & Zinkov, R. (2023). PyMC: A modern, and comprehensive probabilistic programming framework in Python. *PeerJ Computer Science*, 9, e1516.
- Chowdhury, B. D. (2026). Lost in the middle at birth: An exact theory of transformer position bias. *arXiv preprint arXiv:2603.10123*.
- Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), 253–278.
- Dongre, V., Rossi, R. A., Lai, V. D., Yoon, D. S., Hakkani-Tur, D., & Bui, T. (2025). Drift no more? Context equilibria in multi-turn LLM interactions. *arXiv preprint arXiv:2510.07777*.
- Du, Y., Tian, M., Ronanki, S., Rongali, S., Bodapati, S. B., Galstyan, A., Wells, A., Schwartz, R., Huerta, E. A., & Peng, H. (2025). Context length alone hurts LLM performance despite perfect retrieval. *Findings of the Association for Computational Linguistics: EMNLP 2025*, 23281–23298.

- He, Y., Jin, D., Wang, C., Bi, C., Mandyam, K., Zhang, H., Zhu, C., Li, N., Xu, T., Lv, H., Bhosale, S., Zhu, C., Sankararaman, K. A., Helenowski, E., Kambadur, M., Tayade, A., Ma, H., Fang, H., & Wang, S. (2024). Multi-IF: Benchmarking LLMs on multi-turn and multilingual instructions following. *arXiv preprint arXiv:2410.15553*.
- Hoffman, M. D., & Gelman, A. (2014). The No-U-Turn Sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15, 1593–1623.
- Laban, P., Hayashi, H., Zhou, Y., & Neville, J. (2025). LLMs get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*.
- Leviathan, Y., Kalman, M., & Matias, Y. (2025). Prompt repetition improves non-reasoning LLMs. *arXiv preprint arXiv:2512.14982*.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173.
- Mu, N., Lu, J., Lavery, M., & Wagner, D. (2025). A closer look at system prompt robustness. *arXiv preprint arXiv:2502.12197*.
- Seyboldt, A. (2024). *Nutpie: A fast sampler for Bayesian posteriors*. <https://github.com/pymc-devs/nutpie>